

Functions vs. methods

1. In which of the following lines of code is a **method** called?

- ☐ `new_string = "abc".replace("a", "A")`
- ☐ `print(len("b", "a", "c"))`
- ☐ `["a", "b", "c"].append("d")`
- ☐ `{"b", "a", "c"}.add("d")`
- ☐ `print(type(("a", "b", "c")))`
- ☐ `["a", "b", "c"].sort()`
- ☐ `new_list = sorted(["a", "b", "c"])`
- ☐ `joined_str = "".join(["a", "b", "c"])`
- ☐ `splitted = "a b c".split()`

2. In which of the following lines of code is a **method** called?

Given the following code:

```
my_string = "abc"  
my_list = ["a", "b", "c"]  
my_set = {"b", "a", "c"}
```

- ☐ `new_string = my_string.lower()`
- ☐ `new_set = set(my_list)`
- ☐ `my_list.pop()`
- ☐ `my_set.remove("b")`
- ☐ `print(my_string.isupper())`
- ☐ `print(max(my_set))`

Non-mandatory exercises: H

3. In which of the following lines of code is a **list method** called?

Given the following code:

```
my_string = "abc"  
my_list = ["a", "b", "c"]
```

- ☐ my_list.append(my_string)
- ☐ my_list.extend(my_list)
- ☐ "".join(my_list)
- ☐ print(my_list.index("a"))
- ☐ print(type(my_list))

4. In which of the following lines of code is a **string method** called?

Given the following code:

```
my_string = "abc"  
my_list = ["a", "b", "c"]
```

- ☐ new_string = "".join(my_list)
- ☐ print(len(my_string))
- ☐ print(my_string.endswith(my_list[2]))
- ☐ my_list.append(my_string)
- ☐ new_list = "a b c".split()
- ☐ my_list.append(my_string)
- ☐ stripped = " Hello World ! ".strip()

Non-mandatory exercises: H

5. In which of the following lines of code is a **method** called with a **string as an argument**?

(For those familiar with how Python object orientation works: The question does not include sending a string as the argument to the **self** parameter.)

Given the following code:

```
my_string = "abc"
```

```
my_list = ["a", "b", "c"]
```

- ☐ `new_string = "".join(my_list)`
- ☐ `print(len(my_string))`
- ☐ `print(my_string.endswith(my_list[2]))`
- ☐ `my_list.append(my_string)`
- ☐ `new_list = "a b c".split()`
- ☐ `my_list.append(my_string)`
- ☐ `stripped = " Hello World ! ".strip()`

6. In which of the following lines of code is a method **called**?

- ☐ `new_list = "a b c".split()`
- ☐ `["a", "b", "c"].sort`
- ☐ `"abc".isupper()`
- ☐ `new_string = "ABC".lower`

(Note the **called** is emphasised in the text above. If you, for instance, write `print("abc".upper)`, you don't call the method upper, you just print out the method. So it will **not** print out "ABC" as might be expected, but `<built-in method upper of str object at 0x1056cc630>`. To call a method, you need to write:

```
print("abc".upper())
```

This will print out "ABC".)

Iteration

7. What will be printed in the code below?

```
def get_words_with_e(words):
    words_with_e = []
    for word in words:
        if "e" in word:
            words_with_e.append(word)
    return words_with_e

tokenzied_greeting = ["Hello", "there", ",",
    "fellow", "creature", "!", "Early", "bird",
    "catches", "the", "worm"]

print(get_words_with_e(tokenzied_greeting))
```

“Early” will not be saved in `words_with_e`, since it is written with upper case “E”. How would you change the code, to also detect upper case “E”, as in “Early”?

8. The code below will result in `[]` being printed. By moving one code line, it will instead print `['this', 'the']`. Which line, and where would you move it?

```
TH_DETERMINERS = {"the", "that", "these",
    "this", "those"}

def get_th_determiner(words):
    for word in words:
        found_determiners = []
        if word.lower() in TH_DETERMINERS:
            found_determiners.append(word)
    return found_determiners

tokenzied_sents = ["Do", "you", "know", "this", "?",
    "Early", "bird", "catches", "the", "worm", "."]
print(get_th_determiner(tokenzied_sents))
```

Non-mandatory exercises: H

9. What will be printed?

```
def get_max_len(words):
    for word in words:
        max_so_far = 0
        if len(word) > max_so_far:
            max_so_far = len(word)
    return max_so_far

tokenzied_sents = ["Do", "you","know", "this", "?",
    "Early", "bird", "catches", "the", "worm", "."]

print(get_max_len(tokenzied_sents))
```

By moving one code line, it will instead print 7. Which line, and where would you move it?

Non-mandatory exercises: H

10. The code below now prints:

```
hmskt
spnskt
skl

SE_CONSONANTS = {"b", "c", "d", "f", "g", "h", "j",
                 "k", "l", "m", "n", "p", "q", "r", "s", "t",
                 "v", "x", "z", "w"}

def get_pnly_consonants(word):
    only_consonants = []
    for c in word:
        c = c.lower()
        if c in SE_CONSONANTS:
            only_consonants.append(c)
    return "".join(only_consonants)

for word in ["Hmskt", "spanskt", "SKOLA"]:
    print(get_pnly_consonants(word))
```

A. Change the code so that the original upper casing is retained. That is, so that the following is printed:

```
Hmskt
spnskt
SKL
```

Non-mandatory exercises: H

11. The code below will result in `None` being printed. By adding one code line, you can make the code instead print `['this', 'the']`. Which line, and where would you move it?

```
TH_DETERMINERS = {"the", "that", "these",
                  "this", "those"}

def get_th_determiner(words):
    found_determiners = []
    for word in words:
        if word.lower() in TH_DETERMINERS:
            found_determiners.append(word)

tokenzied_sents = ["Do", "you", "know", "this", "?",
                  "Early", "bird", "catches", "the", "worm", "."]

print(get_th_determiner(tokenzied_sents))
```

Non-mandatory exercises: H

12. What is printed?

```
word = "hello"
years = [2002, 2005, 2008]
temperatures = [15.3, 17.1, 25.5, 29.2]

s1 = ["Stockholm", "is", "my", "home", "town"]
s2 = ["I", "am", "going", "to", "Oslo"]
s3 = ["Hello", "there", "world", "?"]
very_small_tokenized_corpus = [s1, s2, s3]

pos_tagged = [("The", "DET"),
              ("sun", "NOUN"),
              ("shines", "VERB"),
              ("today", "ADV")]

for c in word:
    print(c)

for year in years:
    print(year)

for t in temperatures:
    print(t)

for token in s1:
    print(token)

for sentence in very_small_tokenized_corpus:
    print(sentence)

for word_pos in pos_tagged:
    print(word_pos)
```

Non-mandatory exercises: H

13. What is printed?

```
word = "hello"
years = [2002, 2005, 2008]
temperatures = [15.3, 17.1, 25.5, 29.2]

s1 = ["Stockholm", "is", "my", "home", "town"]
s2 = ["I", "am", "going", "to", "Oslo"]
s3 = ["Hello", "there", "world", "?"]
very_small_tokenized_corpus = [s1, s2, s3]

pos_tagged = [("The", "DET"),
              ("sun", "NOUN"),
              ("shines", "VERB"),
              ("today", "ADV")]

for c in word:
    print(type(c))

for year in years:
    print(type(year))

for t in temperatures:
    print(type(t))

for token in s1:
    print(type(token))

for sentence in very_small_tokenized_corpus:
    print(type(sentence))

for word_pos in pos_tagged:
    print(type(word_pos))
```

Non-mandatory exercises: H

14. This code below will result in the error `AttributeError: 'list' object has no attribute 'lower'`. Removig one letter will make it work.

```
TH_DETERMINERS = {"the", "that", "these",
                  "this", "those"}

def get_th_determiner(words):
    found_determiners = []
    for word in words:
        if words.lower() in TH_DETERMINERS:
            found_determiners.append(word)
    return found_determiners

tokenzied_sents = ["Do", "you","know", "this", "?",
                  "Early", "bird", "catches", "the", "worm", "."]

print(get_th_determiner(tokenzied_sents))
```

15. What is printed?

```
s1 = ["Stockholm", "is", "my", "home", "town"]
s2 = ["I", "am", "going", "to", "Oslo"]
s3 = ["Hello", "there", "world", "?"]
very_small_corpus = [s1, s2, s3]

for sentence in very_small_corpus:
    print("-----")
    for word in sentence:
        print(word)
```